



Disciplina: **FGA0314 - Testes de Software**

Terminologia e Conceitos de V&V

Elaine Venson

elainevenson@unb.br

O mundo digital

- Digitalização é essencial para empresas de todos os portes
- Exemplos: saúde, educação, transporte, finanças, entretenimento
- Inovação + versatilidade = sobrevivência no mercado
- Caso Netflix: de DVDs → streaming → conteúdos originais

O papel do software

- A tecnologia sustenta a transformação digital
- Software = motor da inovação
- Qualidade é **não negociável**
- “Comprometer a qualidade é correr uma corrida já perdida” – Gayathri Mohan

Impacto da falta de qualidade

- **CrowdStrike (2024):** atualização defeituosa derrubou sistemas Windows em empresas do mundo todo → voos cancelados, hospitais e bancos afetados
- **Sistema de Pagamentos Digitais da Índia (UPI, 2025):** falha temporária derrubou milhões de transações em serviços públicos e privados, afetando cidadãos e empresas
- **Sistema de Radar (Reino Unido, 2025):** falha de software no sistema de radar do Reino Unido causou a queda das telas de controle de tráfego aéreo, resultando na paralisação de cerca de 150 voos, incluindo Heathrow e Manchester

Falhas em software não são apenas problemas técnicos: afetam economias, governos e cidadãos.

Sectors Affected: ✈ Airlines | 🏥 Healthcare | 💰 Banking | 📺 Media | 🏛 Government



\$5.4 billion
in costs faced by
Fortune 500
companies



\$500 million
cost to Delta;
airline may pursue
legal action



\$12.5 billion
wiped off
CrowdStrike's
stock market value



7,000
NHS patient
appointments
canceled



6 days
before Delta Air Lines'
operations were
back to normal

CrowdStrike Outage in Numbers



Incident Impact
July 19th, 2024



12.75 million
resource-hours
needed for IT teams
to fix machines



\$10
voucher offered to IT
staff by CrowdStrike
as an apology



10%-20%
financial losses
likely covered by
cyberinsurance policies



41%
of impacted
companies
were in the US



10,000
flights canceled
globally



674,476
entities directly
affected worldwide



8.5 million
devices
affected



30%
of Japanese
McDonald's
stores closed

Por que estudar testes?



<https://youtu.be/UdJr2p5RrF0?si=6dpEcUWrcMFSKrgd>

Impacto de VVTS nos projetos de software

“... o custo de garantir que um sistema irá executar de forma satisfatória em termos de requisitos funcionais e não funcionais a partir de atividades de verificação, teste e depuração pode facilmente ir de **50% a 75% do custo total de desenvolvimento.**” – *Hailpern, 2002*

“... em um projeto típico de desenvolvimento de software, **50% do tempo e mais de 50% do custo total** são gastos testando o sistema.” – *Myers, 2004*

“A indústria de software **gasta \$0,50 para cada \$1,00** despendido em desenvolvimento e manutenção nas atividades de encontrar e corrigir erros” – *Capers Jones*

Cultura de qualidade

- Teste = responsabilidade de todo o time
- Qualidade construída ao longo do processo
- Automação + integração contínua = qualidade contínua
- Cada papel (negócio, dev, tester) contribui

“O teste não é uma fase nas equipes ágeis, **o teste é um modo de vida**. Equipes ágeis testam continuamente. É a única maneira de garantir que as *features* implementadas durante uma determinada iteração ou sprint sejam realmente concluídos.” - *Elisabeth Hendrickson, Quality Tree Software*
(tradução livre)

Processos de Engenharia e Verificação

Na engenharia tradicional repetição em altos níveis de automação nas linhas de produção reduz a necessidade de verificação em produtos individuais

Poucos componentes são verificados individualmente

Os produtos finais são testados estatisticamente

Mas alguns produtos requerem técnicas de verificação mais sofisticadas: carros, aviões, computadores...

Verificação e Validação

Conjunto de atividades conduzidas ao longo do ciclo de vida de um projeto de software para garantir a qualidade

Ocorrem em cada estágio do processo de software

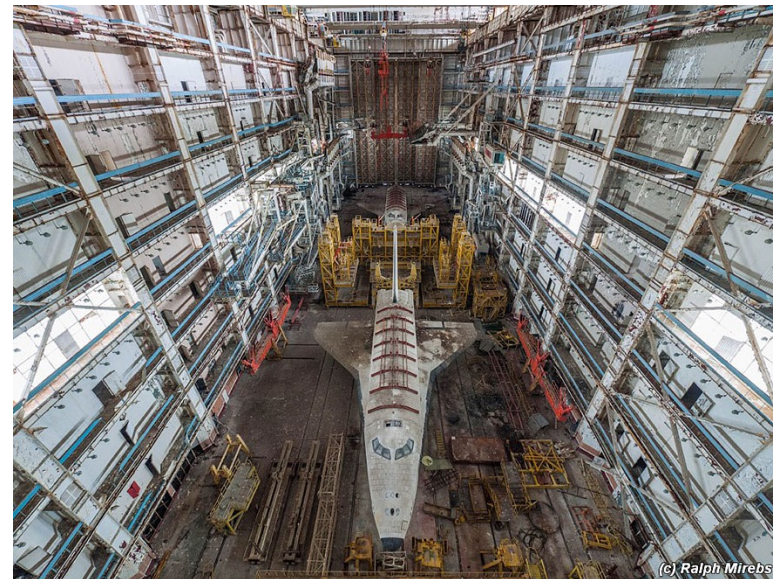
Ocorrem continuamente, “Estilo de vida!”, testes ágeis

Estabelecer confiança de que o software está adequado ao seu propósito!

Processos de Engenharia e Verificação

Produtos produzidos em série x Produtos únicos

A verificação se torna mais difícil de acordo com a complexidade e variedade dos produtos



(c) Ralph Mirebs

Processos de Engenharia e Verificação

Software é um dos produtos de engenharia mais **variáveis e complexos**

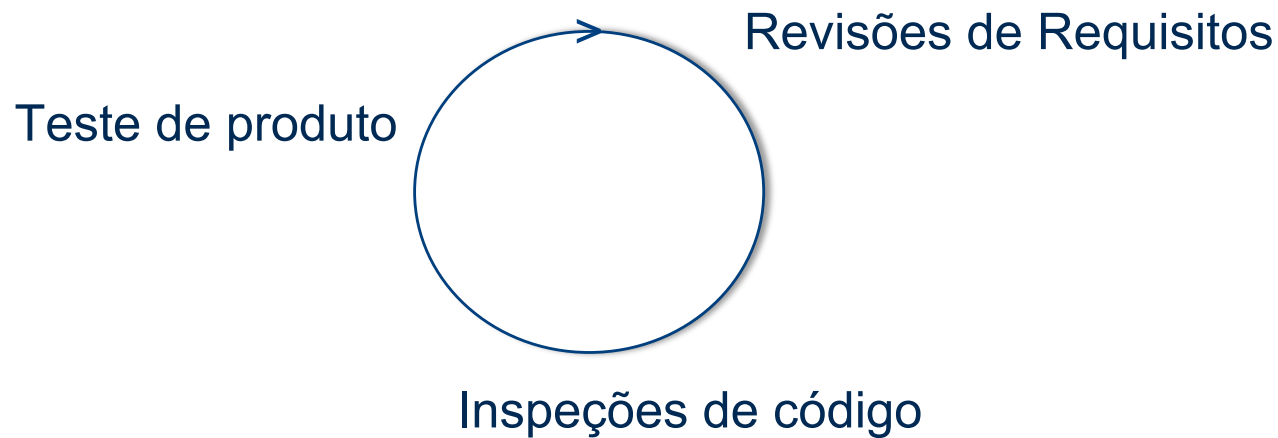
Requisitos de qualidade em **ambientes diferentes** podem ser completamente distintos e incompatíveis

Novas abordagens de desenvolvimento introduzem **novos tipos de defeitos**

O desafio é escolher o **conjunto adequado de técnicas** para alcançar o nível de qualidade necessário, dentro das restrições de custo

Não há “receita de bolo” para resolver o problema da verificação

Verificação e Validação



Verificação x Validação

Validação

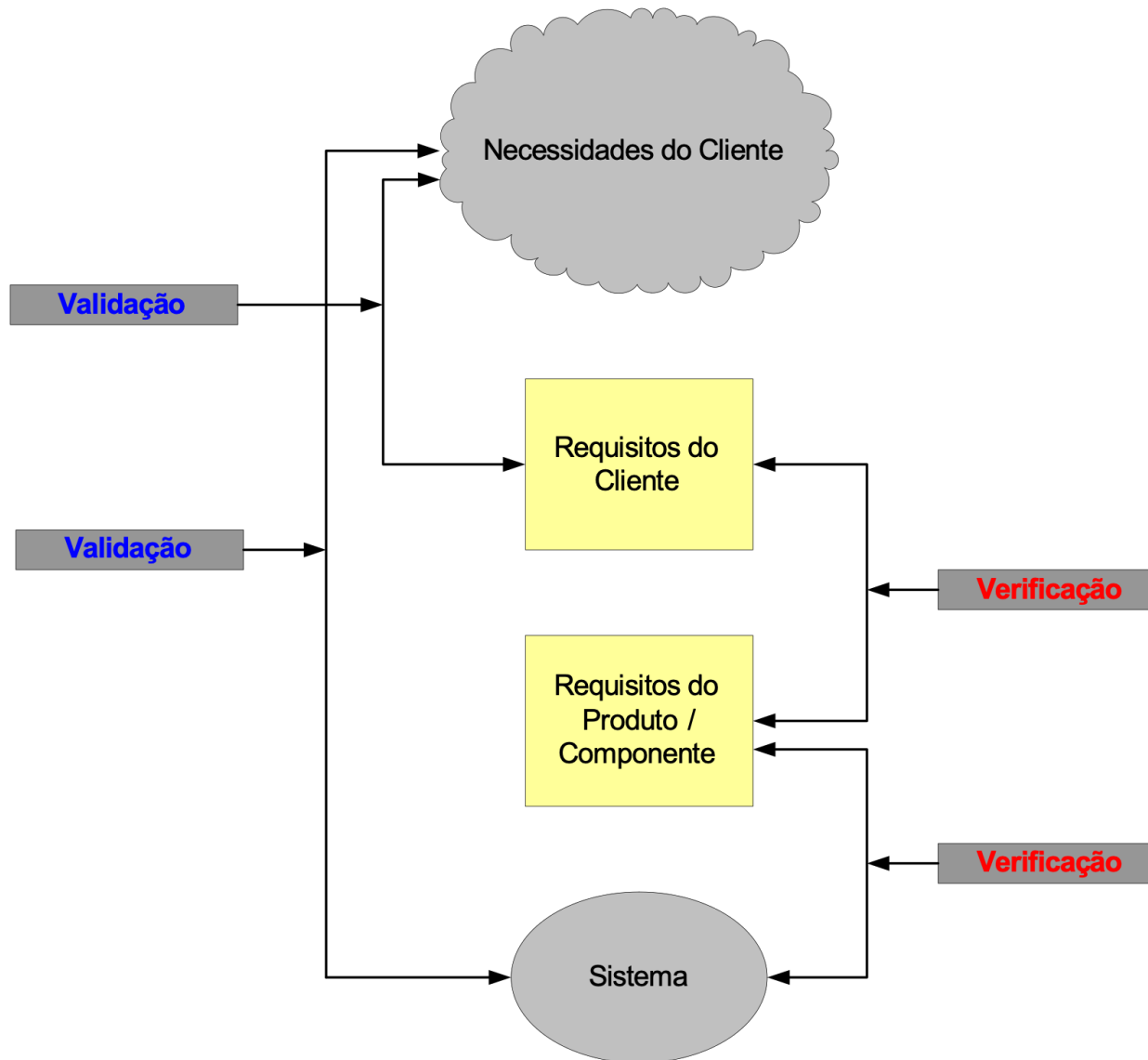
Estamos construindo o produto correto?

Verificação

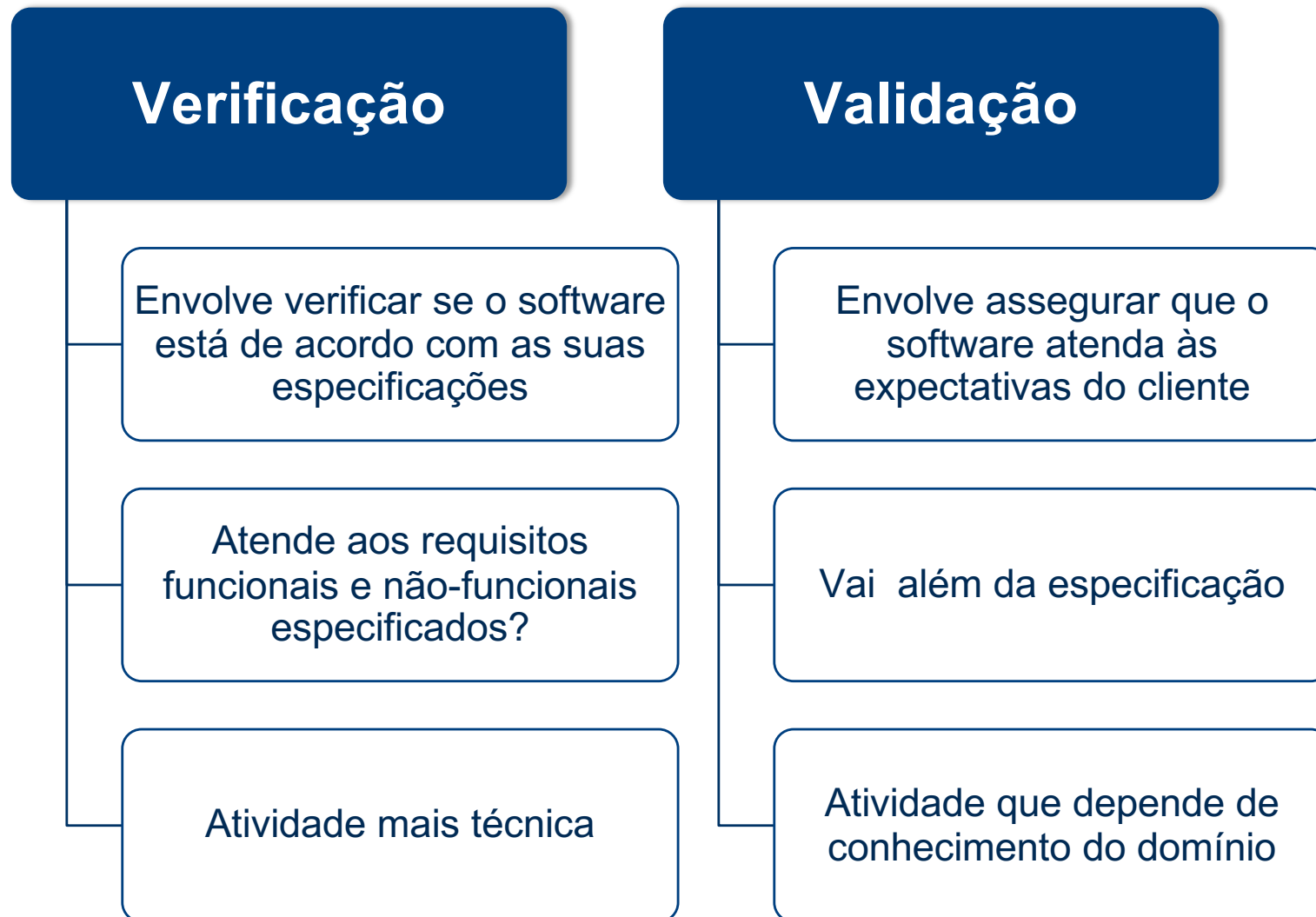
Estamos construindo o produto corretamente?

- Boehm, 1979

Verificação x Validação



Verificação x Validação



Verificação

Por que realizar Verificação?

Grande parte do esforço de um projeto é gasto em **retrabalho**

O custo de correção de defeitos **cresce** a medida que o projeto avança pelas fases de desenvolvimento

Verificação

Definição:

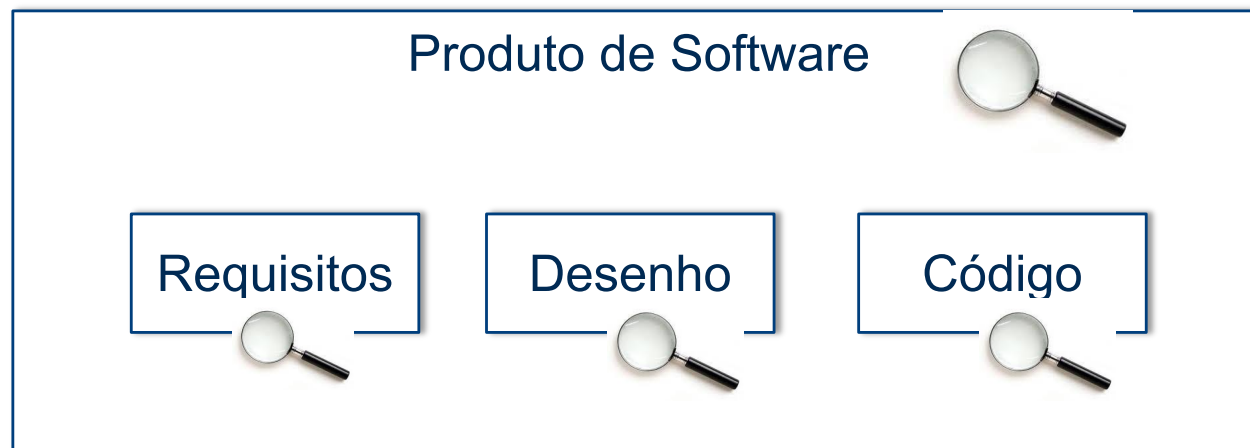
A confirmação, por exame e fornecimento de evidência objetiva, do atendimento aos **requisitos especificados**.

O objetivo da verificação é determinar se os produtos de software de uma atividade atendem completamente aos requisitos ou condições impostas a eles nas atividades anteriores.

[ISO/IEC 12207]

Verificação

Todos os artefatos de um projeto podem ser submetidos à verificação



Verificação

Objetivos:

- assegurar que os requisitos estabelecidos podem ser alcançados

- identificar os requisitos que não podem ser alcançados

- garantir que o software é desenvolvido de forma uniforme

- identificar erros para tomar medidas corretivas o mais cedo possível

- tornar o projeto mais gerenciável

Validação

Frequentemente, a **validação** de software está fortemente associada à **verificação** de software

Normalmente, elas são executadas em conjunto, pois muitas vezes é difícil determinar onde uma começa e onde a outra termina

Validação

De uma maneira geral, pode-se dizer que a **verificação** se preocupa em avaliar se o produto está sendo desenvolvido corretamente

Enquanto a **validação** visa assegurar que se está desenvolvendo o produto correto, isto é, o produto que o cliente deseja

Validação

Definição:

Validação se refere ao processo de examinar um produto para determinar sua conformidade com as **necessidades do usuário**.
[ISO/IEC 12207]

Esta norma define que a validação é feita normalmente no produto final sob condições de operação definidas, podendo, contudo, tornar-se necessária em fases anteriores (protótipos).

Abordagens de V&V

Estáticas:

Não requerem a execução ou existência de um programa executável

Exemplos:

- Revisões

- Inspeções

- Walkthrough

Dinâmicas:

Baseiam-se na execução de um programa

Exemplo:

- Teste de Software

Testes de Software

Definições:

“Teste é o processo de **executar** um programa com a intenção de **encontrar erros**.” - *Myers, 1979*

“Avaliação do software pela **observação de sua execução**.” –
Ammann & Offutt, 2008

Testes de Software

A maioria das definições associa o teste às **técnicas dinâmicas** de Verificação e Validação

Porém, alguns autores definem teste como o processo de encontrar casos em que o programa não atende a sua especificação

Nestes casos, atividades como revisões, inspeções e análise estática de código são também chamadas de teste (teste estático)

Testes de Software

Níveis:

- Teste unitário (pequenas funções)
- Teste de integração (componentes externos)
- Teste de sistema (todo o software)

Caixa-preta x Caixa-branca

Funcionais x Não funcionais (ex.: desempenho, segurança)

Terminologia

Segundo a **ISO/IEC/IEEE 24765:2010(E) Systems and Software Engineering – Vocabulary**:

Engano (*mistake*): ação humana que produz um resultado incorreto

Defeito (*fault, defect*): passo, processo, ou definição de dados incorreto

Comumente, “erro”, “bug” e “defeito” são termos usados para expressar esse significado

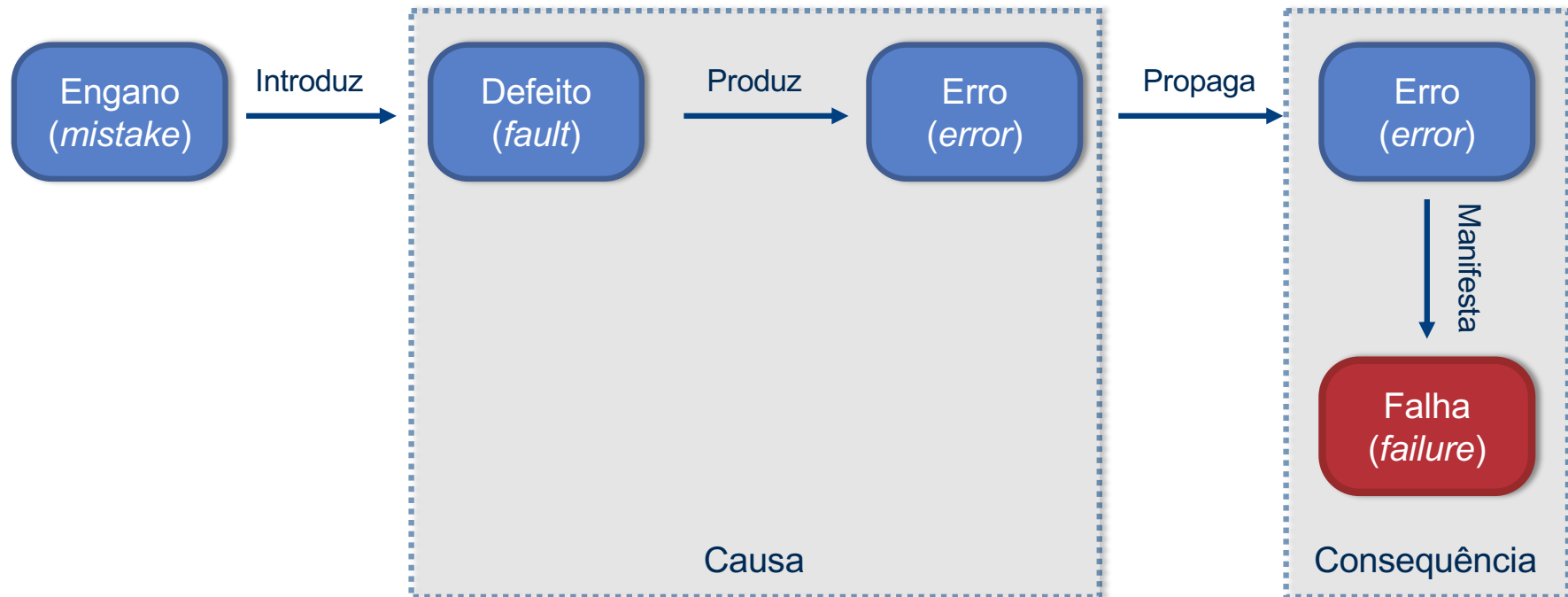
É um conceito estático -> independe de uma execução particular

Erro (*error*): um estado interno inconsistente ou inesperado durante a execução de um programa, ocasionado por um defeito

Falha (*failure*): comportamento externo incorreto, quando o resultado produzido por uma execução é diferente do resultado esperado, em função de um erro

Erro e falha são conceitos dinâmicos

Terminologia



Engano, erro, defeito e falha

Exemplo:

```
public static int numZero (int[] x) {  
    // Effects: if x == null throw NullPointerException  
    //     else return the number of occurrences of 0 in x  
    int count = 0;  
    for (int i = 1; i < x.length; i++)  
    {  
        if (x[i] == 0)  
        {  
            count++;  
        }  
    }  
    return count;  
}
```

Erro, defeito e falha

O defeito está na inicialização da estrutura de repetição FOR, em Java o primeiro elemento do vetor tem índice zero

Para o vetor de entrada [3, 4, 0], o método retorna corretamente o valor 1

Ocorre um erro durante a execução, mas o erro não se manifesta em falha

Para o vetor de entrada [0, 2, 1], o método retorna erroneamente o valor 0

Ocorre o erro e este se manifesta externamente em falha

Outras definições

Falha: execução que resulta de um defeito

Erro: ação humana que origina um defeito

Depuração: o processo de encontrar um defeito dada uma falha

Casos de Testes

Descrição detalhada dos cenários a serem testados

Descrevem:

- as ações que devem ser executadas para verificar se o software se comporta de acordo com o esperado
- os resultados que devem ser obtidos

Caso de Teste

Em geral, um caso de teste contém:

- **Identificador** – um código ou número único
- **Descrição** – o que será testado
- **Pré-condições** – estado inicial necessário para executar o teste
- **Entradas** – valores ou ações que serão aplicados ao sistema
- **Passos de execução** – sequência de ações a serem realizadas
- **Resultado esperado** – o comportamento correto do software
- **Resultado obtido** (após execução)
- **Status** – aprovado (pass) ou falhou (fail)

Exemplo Caso de Teste

- **ID:** CT-01
- **Descrição:** Verificar se função eh_par retorna verdadeiro para número par
- **Pré-condição:** Sistema em execução
- **Entrada:** $n = 4$
- **Passos:** Executar eh_par(4)
- **Resultado esperado:** True
- **Resultado obtido:** (a preencher após execução)
- **Status:** (a preencher: aprovado/falhou)

Definindo Casos de Teste

Requisito:

Dado três valores inteiros positivos que representam os lados de um triângulo, classifique-o como:

- Equilátero (três lados iguais)
- Isósceles (dois lados iguais)
- Escaleno (todos diferentes)
- Ou inválido (não formam triângulo)

➤ Escreva um conjunto de casos de teste para testar este programa!

ID	Entrada	Saída Esperada
CT1	4, 4, 4	equilátero
...	...	...

Execução dos Testes

```
def classifica_triangulo(a, b, c):  
    if a == b and b == c:  
        return "Equilátero"  
    elif a == b or b == c or a == c:  
        return "Isósceles"  
    else:  
        return "Escaleno"
```

Resultado da Execução

CTs executados

CTs que passaram

CTs que falharam

Condição de existência de um triângulo

Para três lados a , b , c , temos que:

- $a + b > c, a + c > b, b + c > a$
- Os lados precisam ser **valores positivos** ($a, b, c > 0$)

Exemplos:

- $a = 3, b = 4, c = 5 \rightarrow$ triângulo escaleno válido.
- $a = 2, b = 2, c = 4 \rightarrow$ não forma triângulo (porque $2 + 2 = 4$).
- $a = 1, b = 2, c = 10 \rightarrow$ não forma triângulo (um lado muito maior que a soma dos outros)

Auto-avaliação

1. Triângulo escaleno válido?
2. Triângulo equilátero válido?
3. Triângulo isósceles válido?
4. 3 permutações do caso anterior?
5. Lado = 0?
6. Lado negativo?
7. Um lado é igual à soma dos outros?
8. 3 permutações do caso anterior?
9. Um lado maior que a soma dos outros?
10. 3 permutações do caso anterior?
11. Todos os lados = 0?
12. Entrada não inteira?
13. Número errado de valores?
14. Para cada caso de teste, o **resultado esperado** está especificado?

Marque um ponto
para cada resposta
SIM!

Auto-avaliação

Profissionais altamente qualificados **acertam em média 7,8 de 14 casos (Myers)**

Testes não garantem descobrir todos os erros

- Mesmo um conjunto completo de casos de teste **não garante que todos os bugs serão encontrados**
- Os casos do exemplo representam **erros que realmente ocorreram** em versões diferentes do programa
- Um teste adequado **deve expor pelo menos esses erros conhecidos**

Casos Válidos

ID	Caso	Entrada (a,b,c)	Resultado Esperado
CT01	Escaleno	3, 4, 5	Escaleno
CT02	Equilátero	3, 3, 3	Equilátero
CT03	Isósceles	3, 3, 2	Isósceles
CT04	Permutações do Isósceles	3,2,3 / 2,3,3	Isósceles

Casos Inválidos

ID	Caso	Entrada (a,b,c)	Resultado Esperado
CT05	Lado = 0	0,2,2	Não é triângulo
CT06	Lado negativo	-1,2,2	Não é triângulo
CT07	Lado = soma dos outros	2,2,4	Não é triângulo
CT08	Permutações CT07	2,4,2 / 4,2,2	Não é triângulo
CT09	Um lado maior que soma dos outros	1,2,5	Não é triângulo
CT10	Permutações CT09	1,5,2 / 5,1,2	Não é triângulo
CT11	Todos lados = 0	0,0,0	Não é triângulo
CT12	Entrada não inteira	3,4,"a"	Erro de entrada
CT13	Número errado de valores	3,4	Erro de entrada

Resumo

Validação (produto correto) x Verificação (desenvolvendo corretamente)

V&V contemplam uma série de técnicas que podem ser aplicadas em todos os pontos do ciclo de vida

O teste de software é a principal técnica de verificação e validação

O teste é uma atividade dinâmica de V&V

Defeito (implementação incorreta), erro (execução inconsistente) e falha (decorrência de um erro)

Testes não garantem descobrir todos os defeitos

Referências

DELAMARO, Marcio. Introdução ao Teste de Software. Disponível em: Minha Biblioteca, (2nd edição). Grupo GEN, 2016.

Myers, G. J. et al, The Art of Software Testing. Ed. John Wiley & Sons, 2012.

Dúvidas?

Elaine Venson

elainevenson@unb.br

